

Reviewer Pack — Plethysm Coefficient Growth and Permanent Circuit Complexity

Entry 743e042ea636 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 13:20:00 UTC

Plethysm Coefficient Growth and Permanent Circuit Complexity

Entry ID: 743e042ea636 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 13:20:00 UTC

1. Conjecture

Field A (mathematical branch): Plethysm of Symmetric Functions **Field B** (complexity object): Boolean Circuit Size for Permanent Polynomial

Statement:

For each $n \geq 1$, the plethysm coefficient of $s_{\{n\}} \circ s_{\{n\}}$ is exponentially larger than the minimal circuit size of the $n \times n$ permanent polynomial over $\text{GF}(2)$.

Rationale (proposer’s reasoning):

The plethysm coefficient captures the multiplicity of irreducible representations in the decomposition, which may reflect the inherent complexity of expressing the permanent as a combination of simpler polynomials. If this coefficient grows exponentially, it suggests that the permanent requires exponentially large circuits, aligning with known lower bounds.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 26a271afb5384e04

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -plethysm symmetric functions permanent polynomial circuit complexity exponential growth -plethysm coefficient permanent polynomial circuit size exponential lower bound -plethysm schur functions gf(2) permanent polynomial circuit complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matrix_mult(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def gaussian_elimination(A, b):
        n = len(A)
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
            for j in range(i+1, n):
                factor = A[j][i] / A[i][i]
                for k in range(i, n):
                    A[j][k] -= factor * A[i][k]
                b[j] -= factor * b[i]
        x = [0] * n
        for i in range(n-1, -1, -1):
            x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
        return x

    def permanent_matrix(n):
```

```

M = [[0] * n for _ in range(n)]
for i in range(n):
    for j in range(n):
        if (i + j) % 2 == 0:
            M[i][j] = 1
return M

def plethysm_coefficient(n):
    # Simplified approximation for demonstration purposes
    return math.factorial(2 * n) / (math.factorial(n) ** 2)

n = random.randint(5, 40)
perm_matrix = permanent_matrix(n)
coeff = plethysm_coefficient(n)

if coeff > 1e10:
    conjecture_holds = False
    counterexample = "plethysm_coefficient_too_large"
else:
    conjecture_holds = True
    counterexample = ""

return {
    "metric_name": "plethysm_coefficient",
    "metric_value": coeff,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_coeff = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_coeff} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_coeff} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"plethysm_coefficient_too_large\" first_failing={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
257874440.0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'plethysm_coefficient too large'
TRIAL: {'metric_name': 'plethysm_coefficient', 'metric_value': 9075135300.0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'plethysm_coefficient too large'}
TRIAL: {'metric_name': 'plethysm_coefficient', 'metric_value': 155117520.0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'plethysm_coefficient too large'}
TRIAL: {'metric_name': 'plethysm_coefficient', 'metric_value': 3432.0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'plethysm_coefficient too large'}
TRIAL: {'metric_name': 'plethysm_coefficient', 'metric_value': 126410606437752.0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'plethysm_coefficient too large'}
TRIAL: {'metric_name': 'plethysm_coefficient', 'metric_value': 8233430727600.0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'plethysm_coefficient too large'}
TRIAL: {'metric_name': 'plethysm_coefficient', 'metric_value': 48620.0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'plethysm_coefficient too large'}
TRIAL: {'metric_name': 'plethysm_coefficient', 'metric_value': 6.892620648693261e+21, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'plethysm_coefficient too large'}
TRIAL: {'metric_name': 'plethysm_coefficient', 'metric_value': 601080390.0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'plethysm_coefficient too large'}
TRIAL: {'metric_name': 'plethysm_coefficient', 'metric_value': 4.425125402768368e+20, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'plethysm_coefficient too large'}
TRIAL: {'metric_name': 'plethysm_coefficient', 'metric_value': 35345263800.0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'plethysm_coefficient too large'}
TRIAL: {'metric_name': 'plethysm_coefficient', 'metric_value': 2333606220.0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'plethysm_coefficient too large'}
RESULT: FALSIFIED counterexample="plethysm coefficient too large" first failing seed=11
```

8. Critic adversarial review

Critic verdict: CHALLENGE

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

```
parse fail | next: NONE
```

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111729
2	propose	ollama_remote	qwen3:8b	0	0	90243
3	preregistration	ollama_remote	qwen3:8b	0	0	24030
4	novelty	ollama_remote	qwen3:8b	0	0	20171
5	novelty	ollama_remote	qwen3:8b	0	0	14539
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13540
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9277
8	critic	ollama_remote	qwen3:8b	0	0	29742
9	judge	ollama_remote	qwen3:8b	0	0	21156

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 334428 ms total latency. Provider mix: {'ollama remote': 9}

(full prompt+response transcripts available in `research/audit/743e042ea636.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/743e042ea636.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/743e042ea636.tar.gz` (if generated)